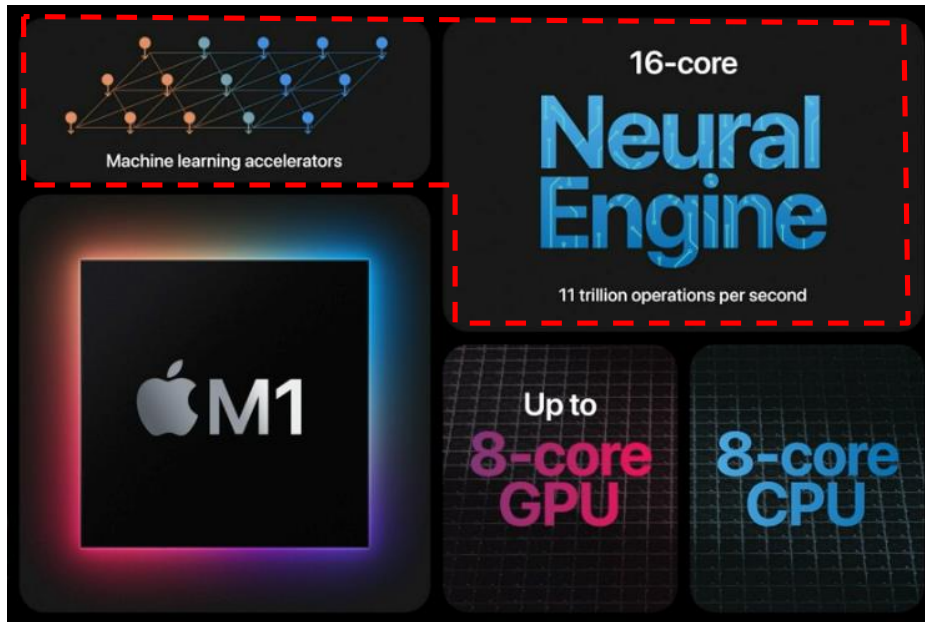


High-Level Synthesis Programming with FPGAs

Choi, Young-kyu

- In this course, we will learn:
 - How to program a high-level synthesis (HLS) code
 - How to optimize an HLS code
 - How to generate an FPGA bitstream from HLS code and run it on an FPGA board
 - How to use Amazon Web Service F1 Instance
- This course is NOT about:
 - How to develop an HLS compiler
 - How to build an FPGA

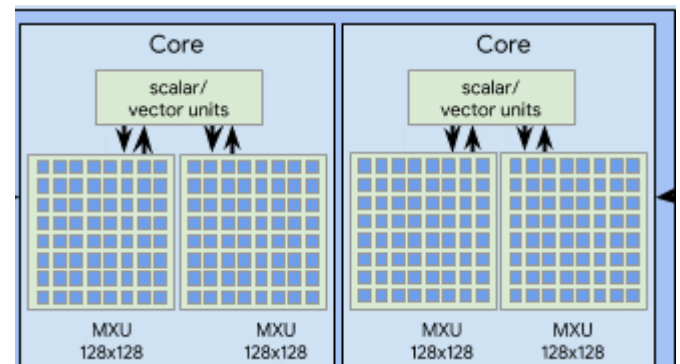
Customization in the Processor World



Apple M1



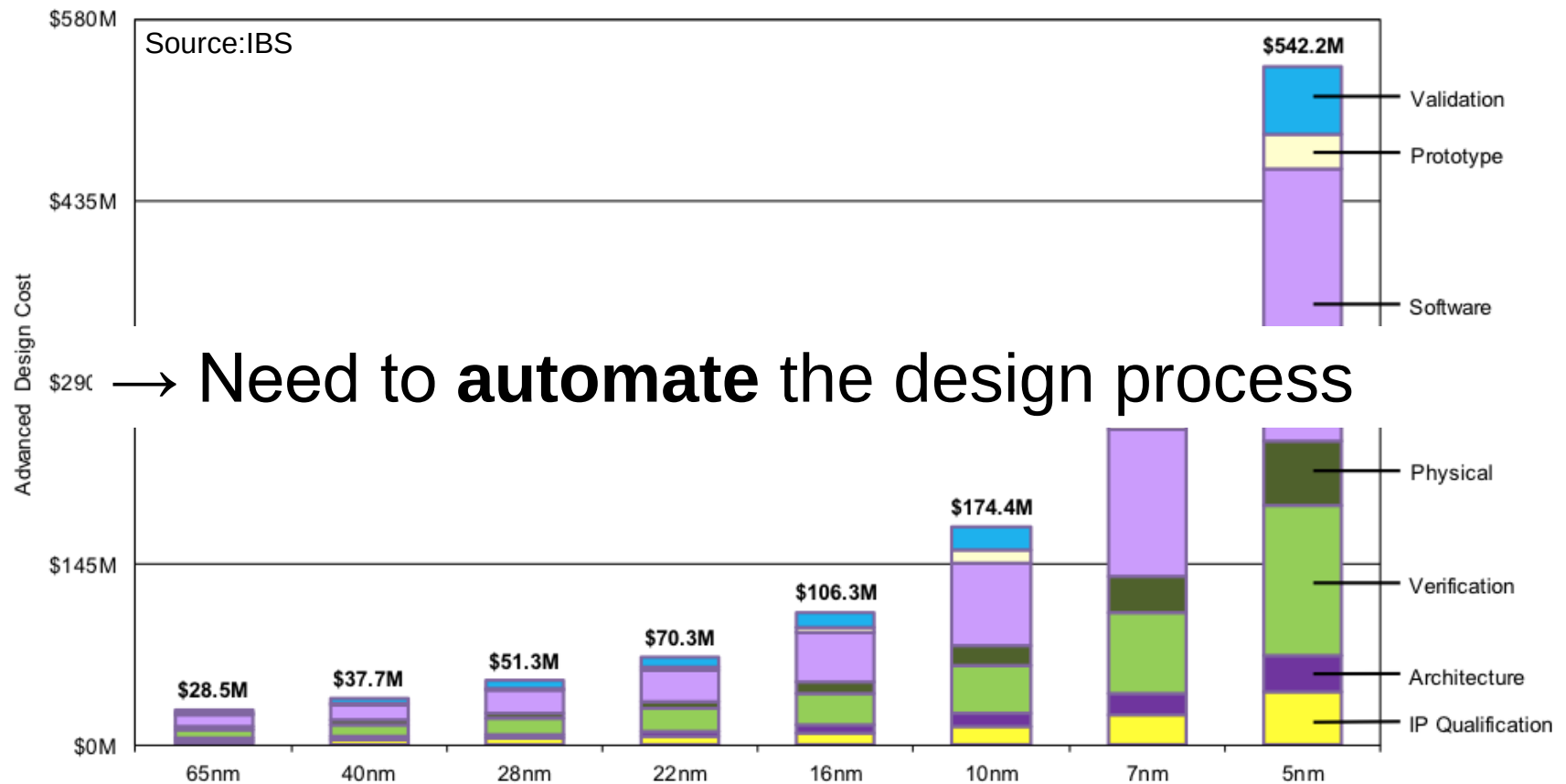
Intel Loihi



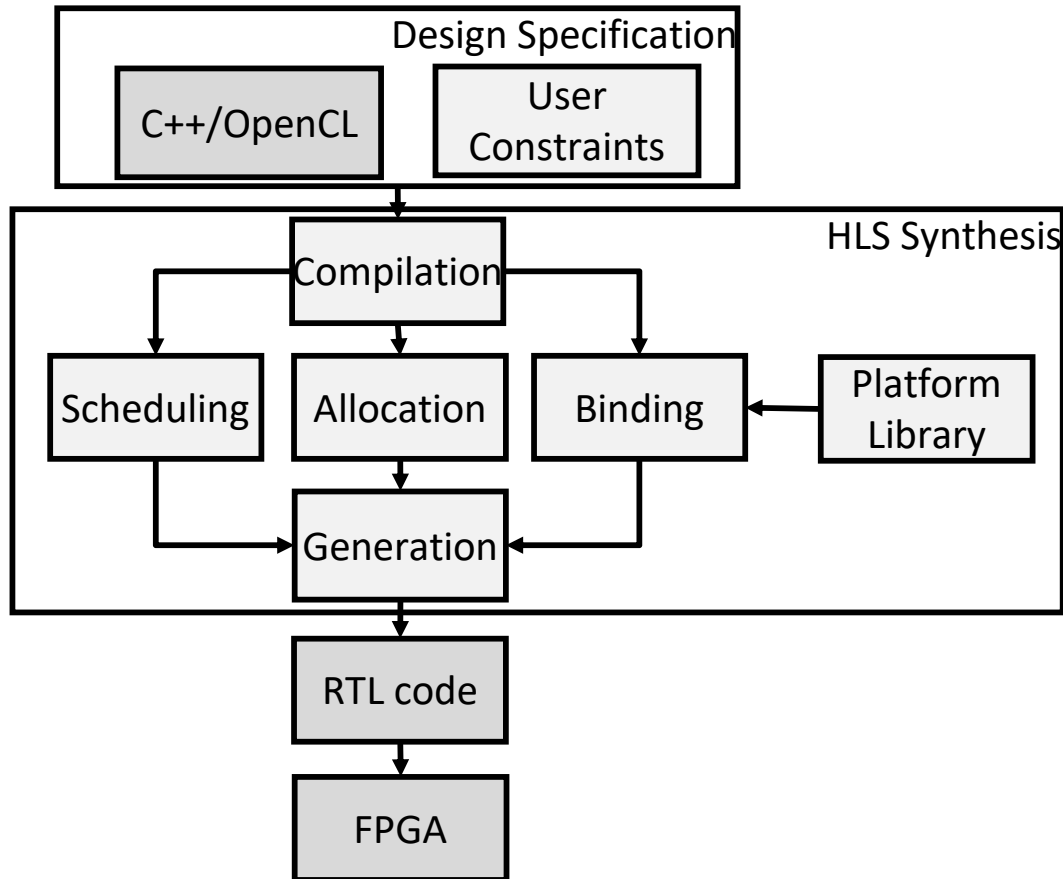
Google TPU

Is It Easy to Design a Customized Accelerator?

- Increasing chip fabrication cost



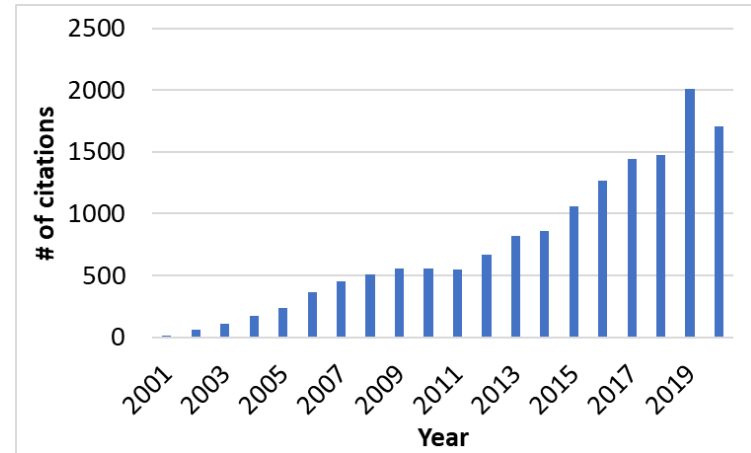
Introducing High-Level Synthesis (HLS)



- Design in high-level language (C++ or OpenCL)
- Automatically determines the microarchitecture
- Generates low-level register-transfer level (RTL) designs
- Increases productivity, faster time-to-market

Popularity of HLS Tools

- Citation count of “high-level synthesis” in Web of Science



Intel® HLS Compiler Success Story

Horizon Robotics is partnering with Intel to develop embedded AI applications. Learn how Horizon Robotics is using the Intel® Arria® 10 FPGA and Intel® HLS Compiler to do real-time pixel level segmentation for 3D semantic modeling and localization.

Working smarter, not harder:
NVIDIA closes design
complexity gap with HLS

Success Story

Google develops WebM
video decompression
hardware IP using High-
Level Synthesis

High-Level Synthesis (HLS) vs Hardware Description Language (HDL)

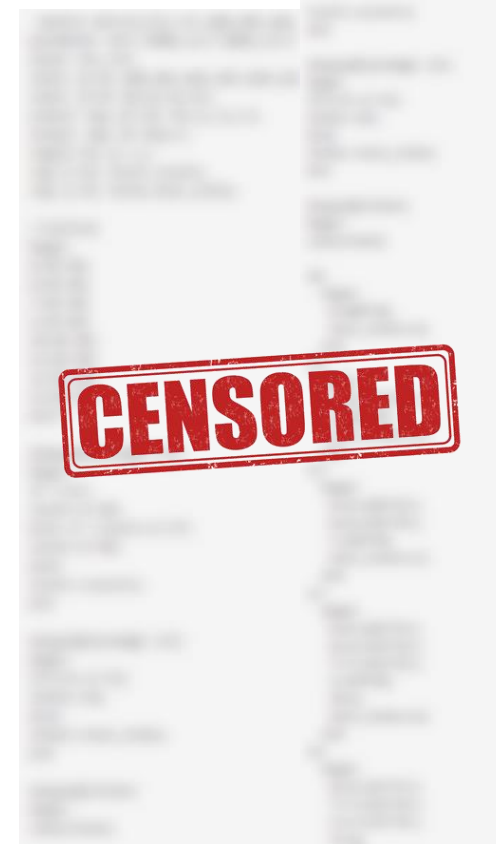
- Example: Matrix-vector multiplication

<Verilog(HDL)>

<HLS>

```
void mv(ap_uint<512> *A, int *y){  
#pragma HLS INTERFACE m_axi port=A bundle=gmem0  
#pragma HLS INTERFACE m_axi port=y bundle=gmem1  
    int psum;  
    ...  
  
    for(row=0; row<N; row++){  
        for(col1=0; col1<N/16; col1++){  
#pragma HLS pipeline II=1  
            for(col2=0; col2<16; col2++){  
                psum += A[col1*16+col2] * x[col1*16+col2];  
            }  
        }  
        y[row] = psum;  
    }  
}
```

~20 lines of code



1000s of lines of code

HLS vs RTL Result

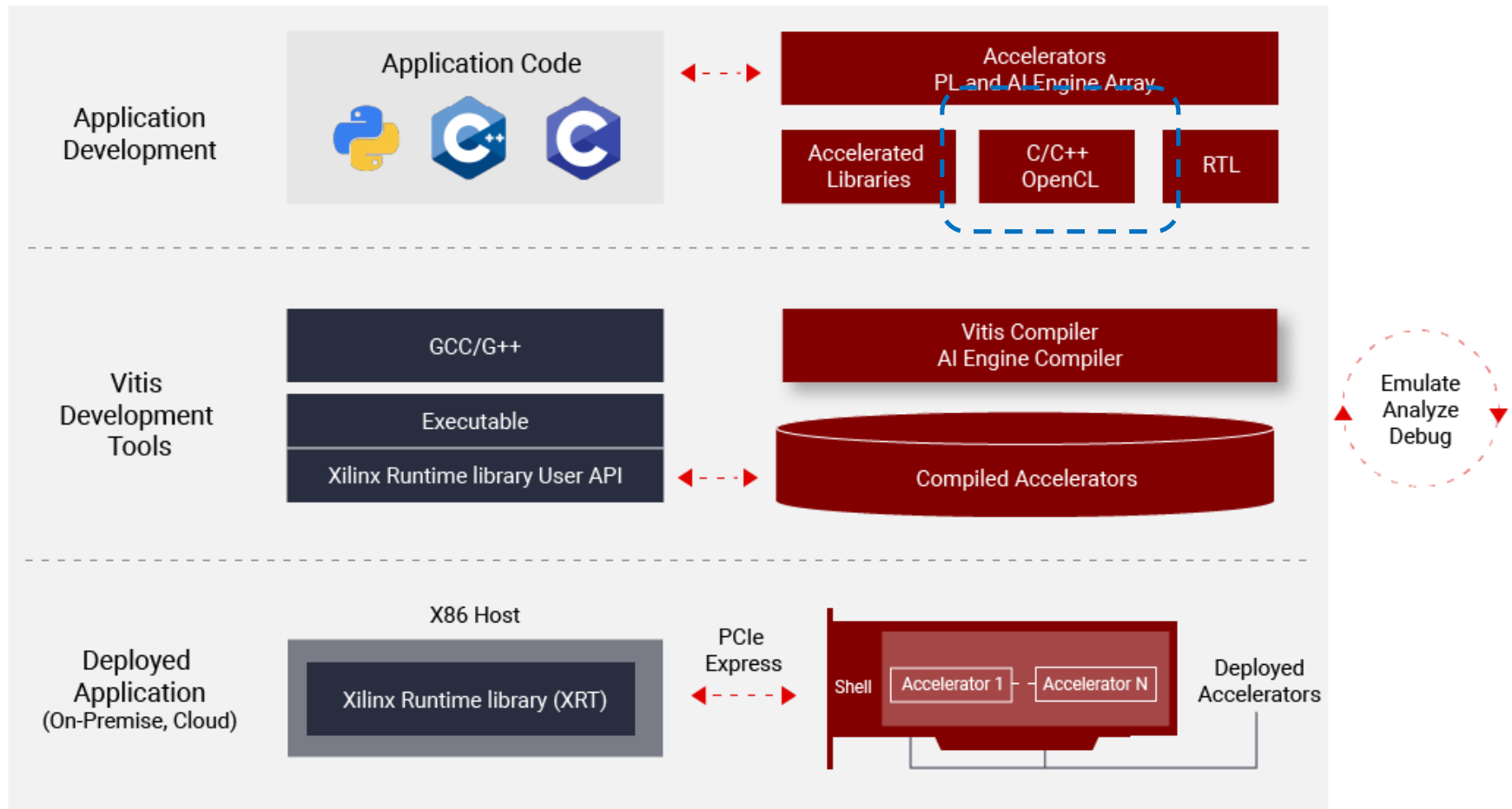
- Example: 2D discrete cosine transform (DCT)

HLS			
Person #	Area (LUTs)	Freq. (MHz)	Speed*
1	1 860	214	258
2	3 161	101	588
3	2 814	211	675
4	2 273	167	972
5	2 768	137	797
6	2 463	211	750
Avg.	2 557	174	673
RTL			
1	4 000	145	197
2	2 068	108	192
3	1 292	308	458
4	4 431	148	499
5	2 066	137	122
6	2 722	149	121
Avg.	2 763	166	265

HLS AND RTL PRODUCTIVITY					
Person #	HLS		RTL		HLS/RTL Quality Ratio
	Hours	Quality*/Hours	Hours	Quality*/Hours	
1	2	80	4	14	5.9×
2	4	44	9	11	4.1×
3	12	19	21	17	1.1×
4	9	47	18	6	7.6×
5	5	60	9	6	9.4×
6	20	15	26	2	9.0×
Avg.	9	44	14	9	6.2×

Lahti et al., “Are We There Yet? A Study on the State of High-Level Synthesis,” TCAD’18.

Xilinx Vitis Software Platform



<https://www.xilinx.com/products/design-tools/vitis/vitis-platform.html>

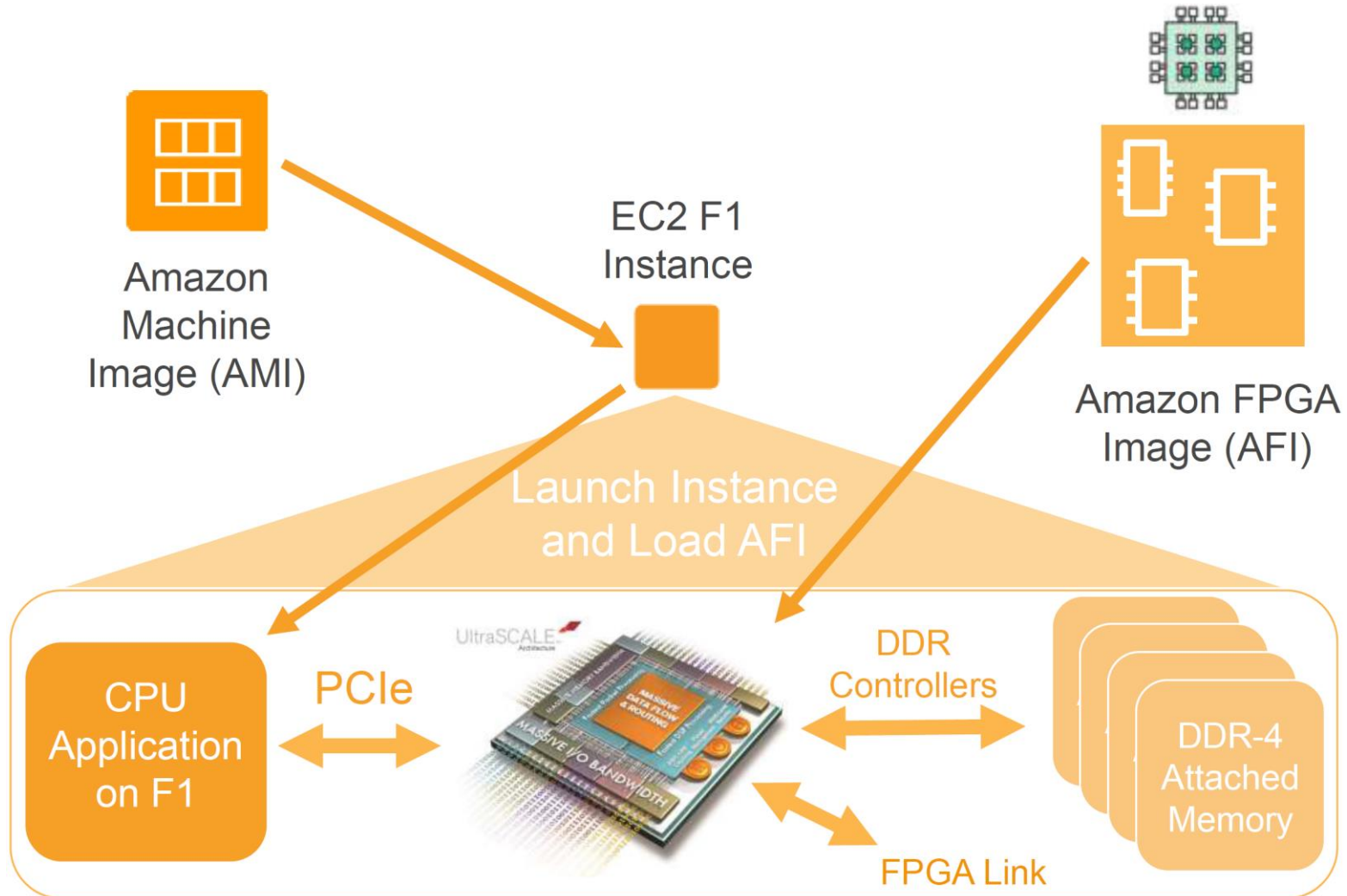
Vitis High-Level Synthesis



- Released by Xilinx in 2020
 - Replaces Vivado HLS
- Compile, simulates, and debugs C, C++, OpenCL functions
- Generates RTL code
- About 1/3 of HLS papers use Vivado HLS**

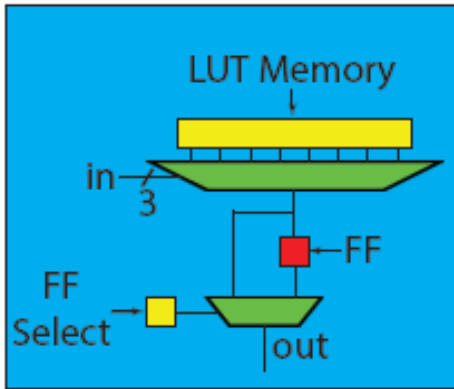
** Lahti et al., “Are We There Yet? A Study on the State of High-Level Synthesis,” TCAD’18.

Platform: Amazon Web Service F1

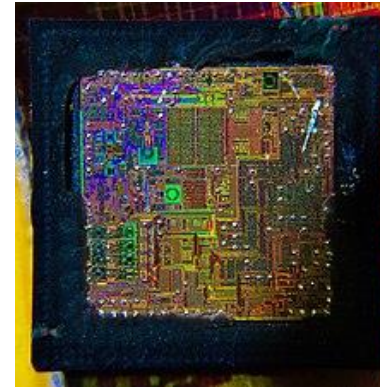


Why use FPGAs for HLS programming?

Field-Programmable Gate Array (FPGA)



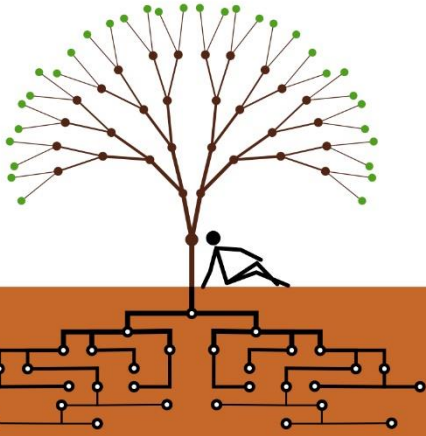
Application-Specific Integrated Circuit (ASIC)



[https://en.wikipedia.org/wiki/Die_\(integrated_circuit\);](https://en.wikipedia.org/wiki/Die_(integrated_circuit);)

Textbook

Parallel Programming for FPGAs
The HLS Book



Ryan Kastner
Janarбек Matai
Stephen Neuendorffer

- Available online:
 - <http://kastner.ucsd.edu/hlsbook/>
- Authors:
 - Ryan Kastner (UCSD)
 - Janarбек Matai (UCSD)
 - Stephen Neuendorffer (Xilinx)



Attribution 4.0 International (CC BY 4.0)

Acknowledgement

- Supported by AWS Korea
 - Thanks to Junghan Kim, Yoocheon Won
- This course has been developed in collaboration with:
 - Jungwook Choi (Hanyang University)
 - Jason Cong (University of California, Los Angeles)
 - Zhenman Fang (Simon Fraser University)
 - Peipei Zhou (University of Pittsburgh)
- Thanks to
 - Inseong Jun (Inha University)

<HLS Programming with FPGAs>

The HLS Book

Chapter 1

Introduction

Choi, Young-kyu

Learning Objectives

- Explain the basic architecture of FPGAs
- Characterize the performance of an HLS design
- Explain the difference between how CPUs and FPGAs do computation

1.1 High-Level Synthesis

- High-level synthesis (HLS) tool
 - Takes high-level language (e.g. C/C++) design and generates detailed RTL micro-architecture (e.g. Verilog)
 - Enables a designer to focus on architectural questions rather than individual registers and cycle-to-cycle operations
- Output of HLS (RTL files) can be synthesized into FPGA bitstream for on-board execution

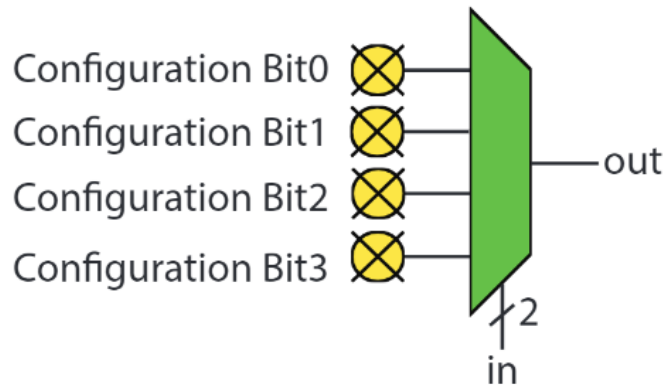
- HLS can automatically:
 - Analyze and exploit concurrency in an algorithm
 - Insert registers in critical paths and achieve desired clock frequency
 - Generate control logic that directs the datapath
 - Implement interface to the rest of the system
 - Map data onto storage elements
 - Map computation onto logic elements

- Limitation of HLS
 - No dynamic memory allocation (e.g. malloc, free, new)
 - No system calls (e.g. exit, printf)
 - No recursive function calls
 - Limited pointer usage
 - Limited use of standard libraries

1.2 FPGA Architecture

- Array of programmable **logic blocks** and **memory elements** connected together using **programmable interconnect**

a) Lookup Table (LUT)

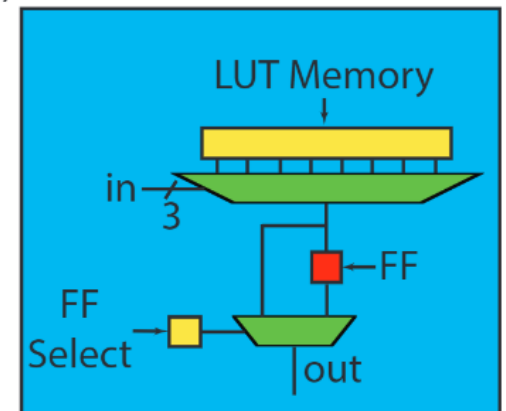


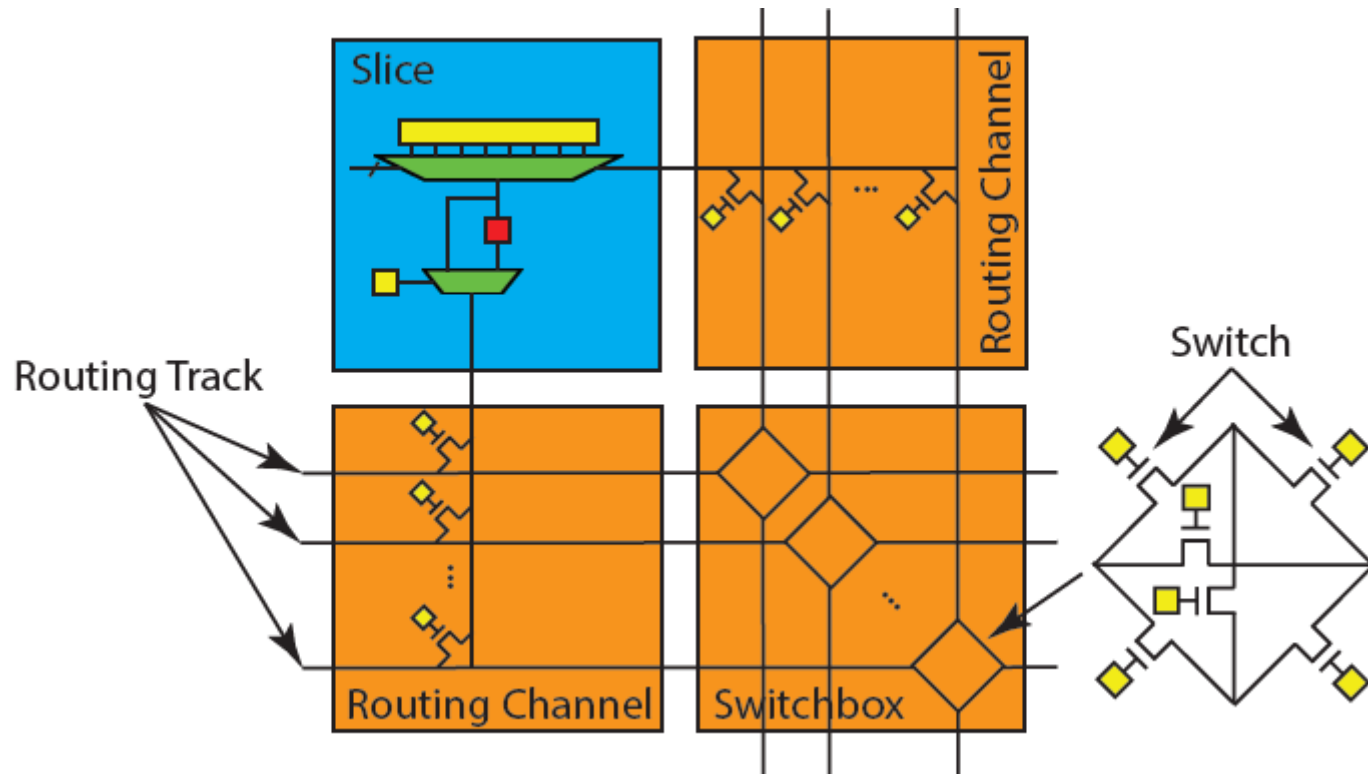
b) Ex) LUT of AND gate

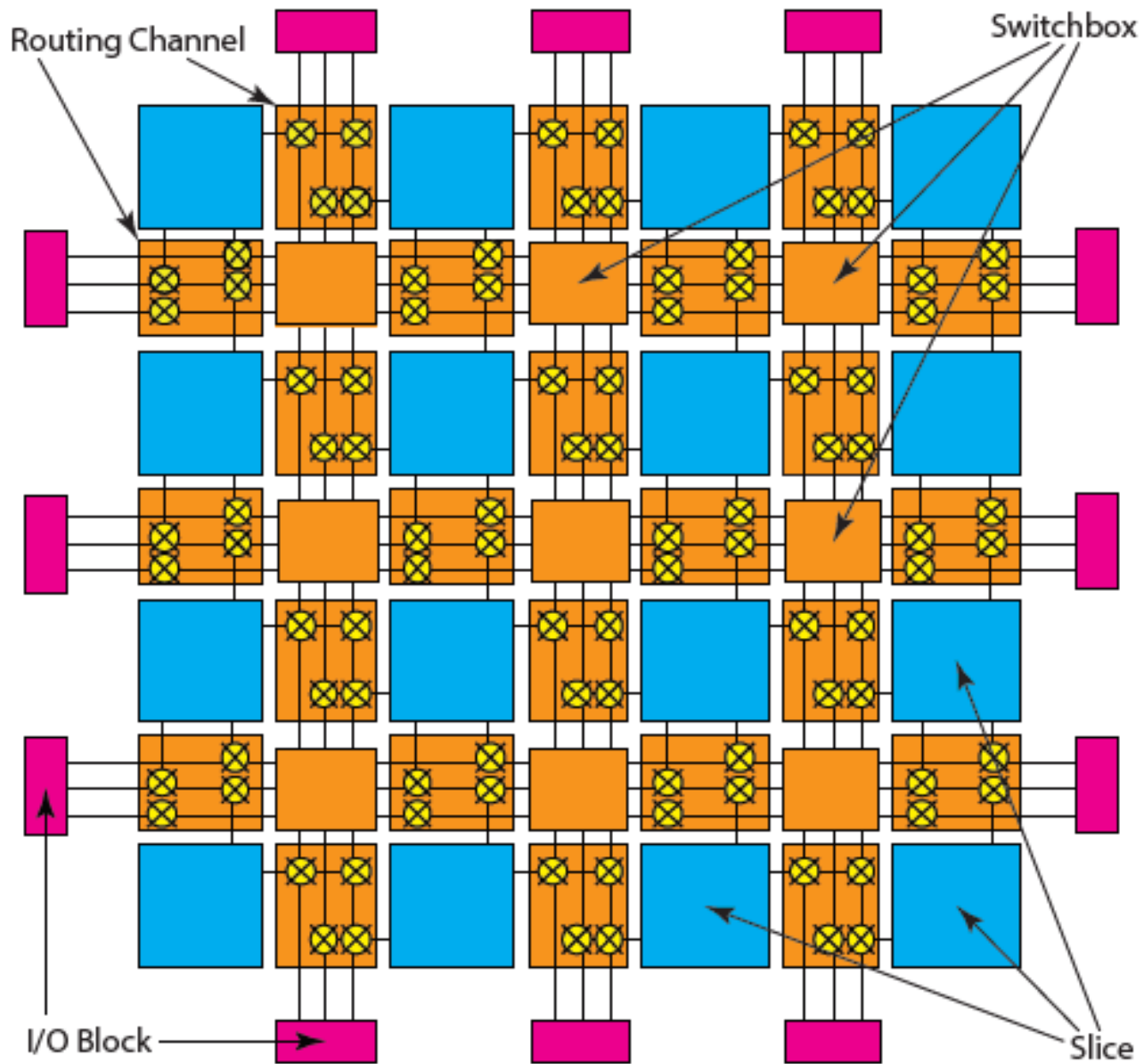
in[1]	in[0]	out
0	0	0
0	1	0
1	0	0
1	1	1

out = in[1] & in[0]

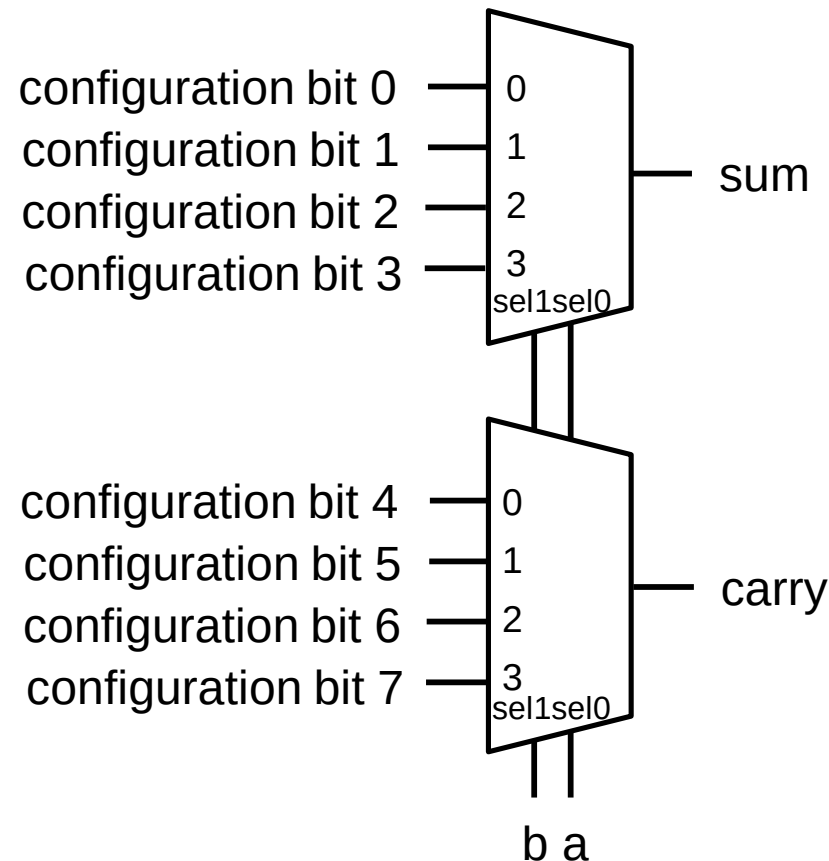
c) Slice



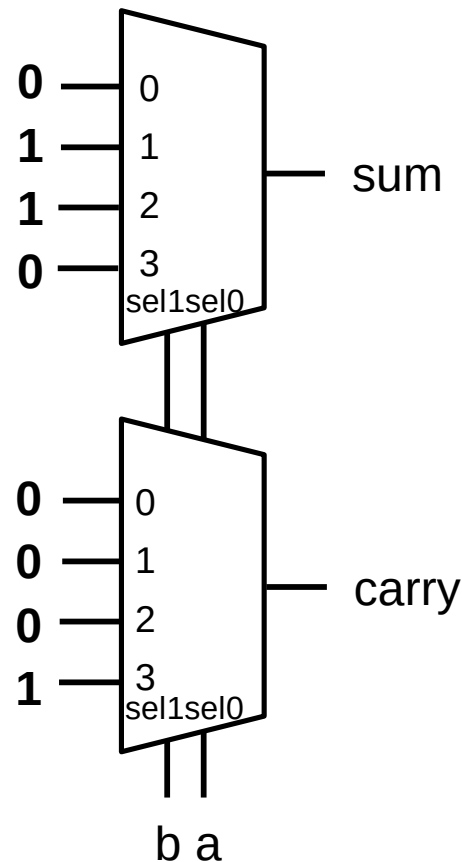




- Exercise
 - Design a half adder by assigning values to the LUT configuration bits



- Solution to exercise
 - Design a half adder by assigning values to the LUT configuration bits



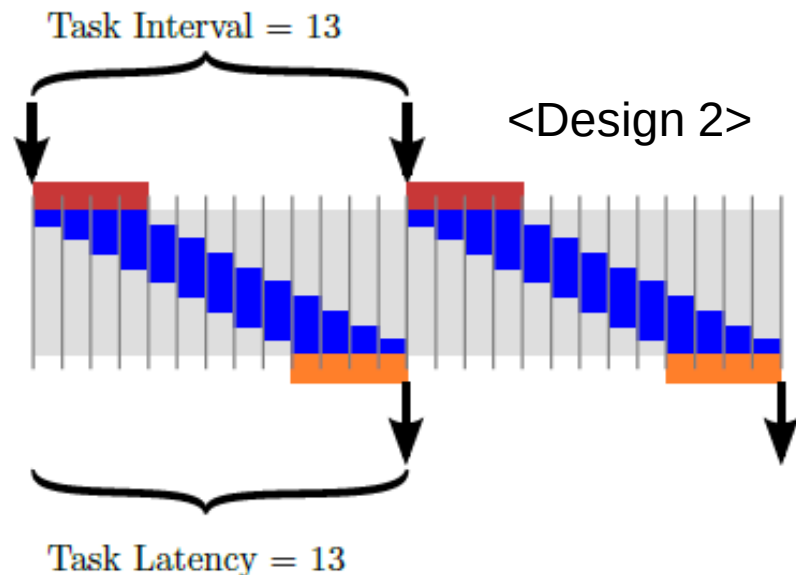
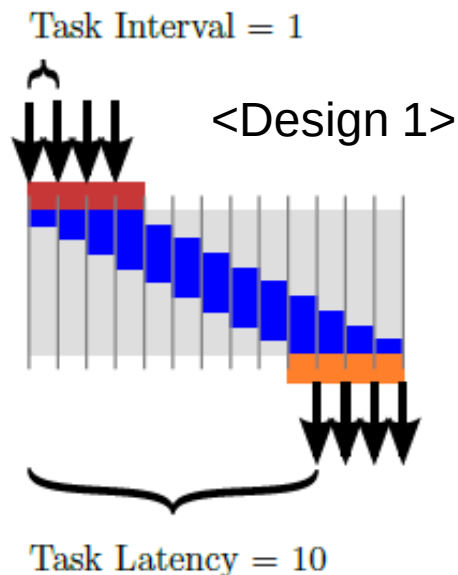
FPGA Memory Resource

- Flip-Flop (FF)
- Block RAM (BRAM)
 - May be configured as 32K x 1b, 16K x 2b, 8K x 4b, etc
- External memory (DRAM)

	External Memory	BRAM	FFs
count	1-4	thousands	millions
size	GBytes	KBytes	Bits
total size	GBytes	MBytes	100s of KBytes
width	8-64	1-16	1
total bandwidth	GBytes/sec	TBytes/sec	100s of TBytes/sec

1.4 Design Optimization

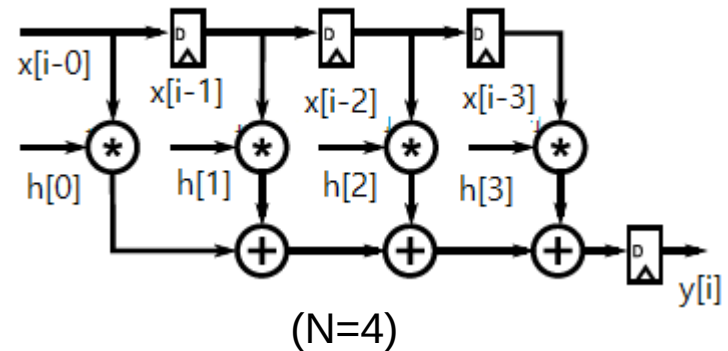
- Performance Characterization
 - Task latency: Time between when a task starts and when it finishes
 - Task interval (= Initiation Interval = II): Time between when one task starts and the next one starts



Area/Throughput Tradeoff

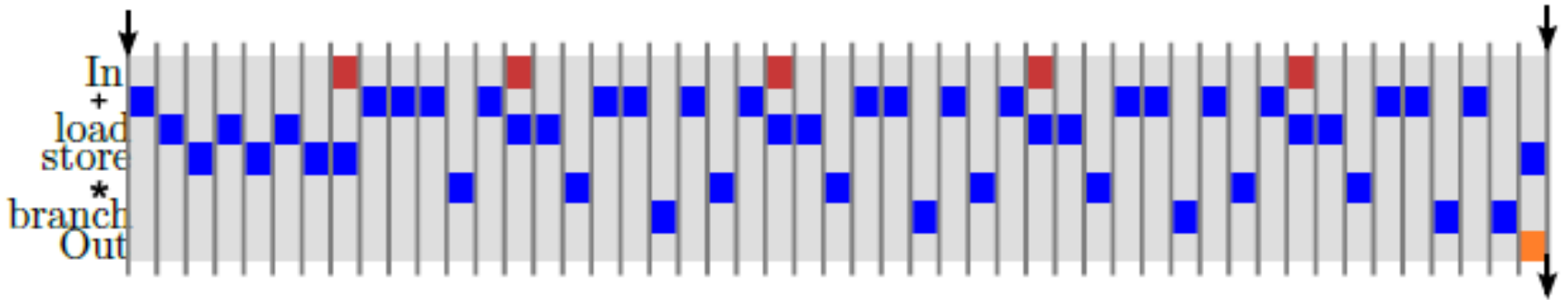
- Example - Finite Impulse Response (FIR) filter

$$y[i] = \sum_{j=0}^{N-1} h[j] \cdot x[i-j]$$

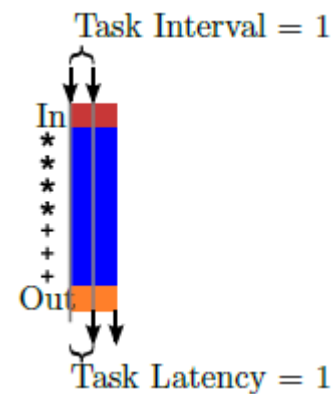
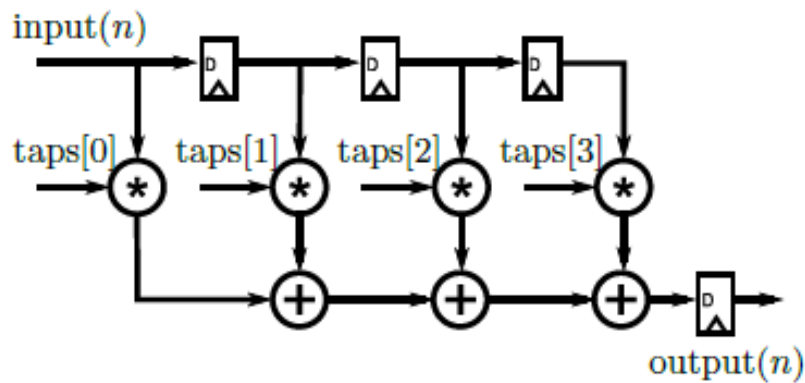


$$y[i] = \sum_{j=0}^{N-1} h[j] \cdot x[i - j]$$

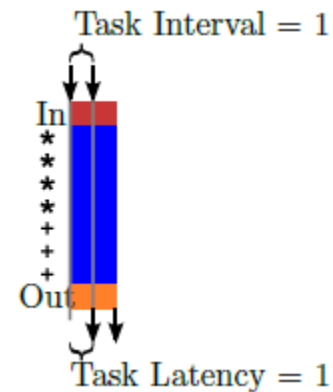
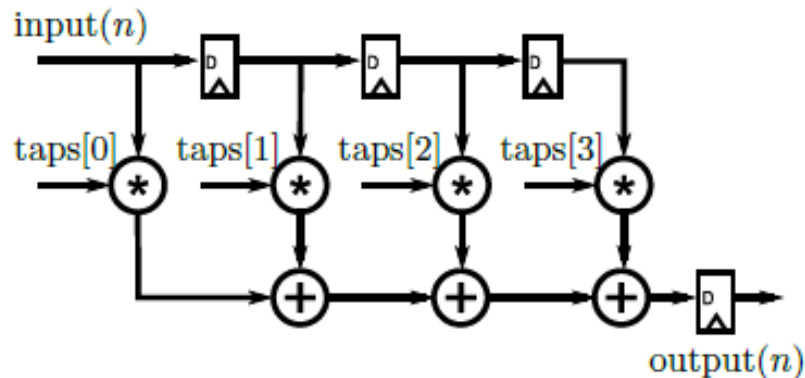
- RISC: * IF, * ID, * EX, * M, * WB, + IF, + ID, + EX, ...



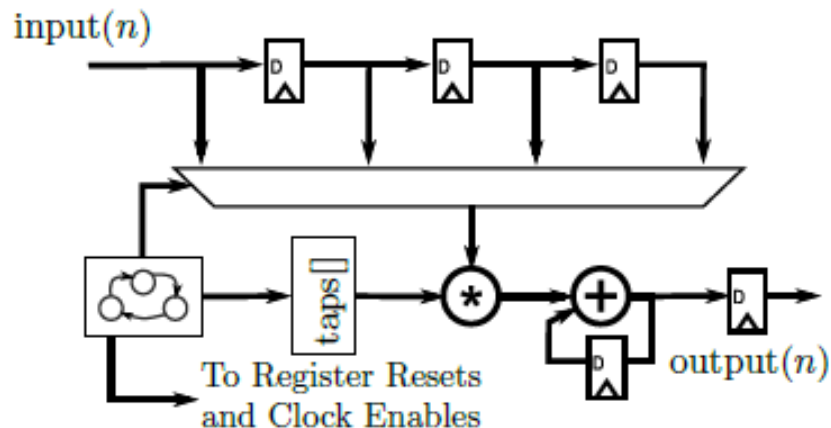
- HLS: creates function pipeline



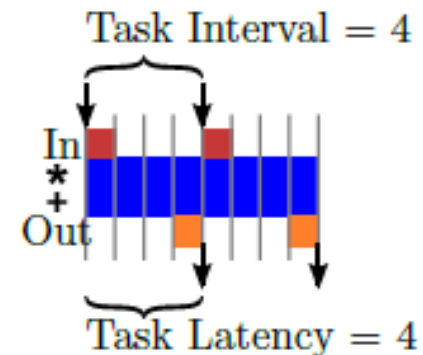
- HLS “one sample per clock”:



- HLS “one tap per clock” :



Tradeoff?



Summary

- You can customize the computation of FPGAs using look-up tables (LUTs)
- FPGA has customizable memory resource and programmable interconnect
- Task latency vs task interval (initiation interval)
- HLS creates function pipeline